



- 1 Introduction
- 2 Monte-Carlo Learning
- 3 Temporal Difference Learning
- 4 TD( $\lambda$ )

---

Some slides are from: David Silver (DeepMind), Katerina Fragkiadaki (CMU), Emma Brunskill (Stanford).

- ① Introduction
- ② Monte-Carlo Learning
- ③ Temporal Difference Learning
- ④ TD( $\lambda$ )

# Outline

- Last Lecture
  - Planning by dynamic programming
  - Solve a known MDP
- This Lecture
  - Model-free prediction
  - Estimating the value function of an unknown MDP
- Next Lecture
  - Model-free control
  - Optimize the value function of an unknown MDP

# Coming Up

- So far, to estimate value functions, we have been using dynamic programming with **known reward and dynamics** functions:

$$V_{\pi,[k+1]}(s) = \sum_a \pi(a|s) \left( r(s, a) + \gamma \sum_{s'} p(s'|s, a) V_{\pi,[k]}(s') \right), \quad \forall s$$

$$V_{[k+1]}(s) = \max_a \left( r(s, a) + \gamma \sum_{s'} p(s'|s, a) V_{[k]}(s') \right), \quad \forall s$$

- Next: estimate value functions and policies **from interaction experience, without known dynamics**. **Q: How?**
- **Sampling!**. Instead of using probability distributions to compute expectations, we estimate expectations by averaging sampled returns.

- 1 Introduction
- 2 Monte-Carlo Learning
- 3 Temporal Difference Learning
- 4 TD( $\lambda$ )

# Monte-Carlo (MC) Reinforcement Learning

- MC methods learn directly from episodes of experience
  - Experience  $\rightarrow$  values, policy
- MC is **model-free**: no knowledge of MDP transitions/rewards
- MC learns from **complete** episodes: **no bootstrapping** (explain later)
- MC uses the simplest possible idea: value = mean return
- **Caveat**: can only apply MC to **episodic MDPs**
  - All episodes must terminate

# Monte-Carlo Policy Evaluation

- **Goal:** Learn  $V_\pi$  from episodes of experience under policy  $\pi$ :

$$s_1, a_1, r_1, s_2, a_2, r_2, \dots, \sim \pi$$

- Recall that the **return** is the total discounted reward

$$G_t = r_t + \gamma r_{t+1} + \dots + \gamma^{T-t} r_T$$

- Recall that the **value function** is the expected return

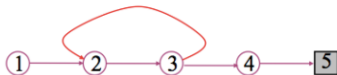
$$V_\pi(s) = \mathbb{E}[G_t | s_t = s]$$

- Monte-Carlo policy evaluation uses **empirical mean return** (vs. expected return)



# Monte-Carlo Policy Evaluation

- **Goal:** learn  $V_\pi(s)$  from episodes of experience under policy  $\pi$
- Idea: Average returns observed after visit to  $s$



- **Every-Visit MC**: average returns for every time  $s$  is visited in an episode
- **First-Visit MC**: average returns only for the first time  $s$  is visited in an episode
- Both converge asymptotically based on the **law of large numbers**

# First-Visit/Every-Visit Monte-Carlo Policy Evaluation

- **First time-step**  $t$  that the state  $s$  is visited in an episode,
- Increment count  
 $N(s) \leftarrow N(s) + 1$
- Increment total return  
 $S(s) \leftarrow S(s) + G_t$
- Value is estimated by mean return  $V(s) = S(s)/N(s)$
- By law of large numbers,  
 $V(s) \rightarrow V_\pi(s)$  as  $N(s) \rightarrow \infty$

- **Every time-step**  $t$  that the state  $s$  is visited in an episode,
- Increment count  
 $N(s) \leftarrow N(s) + 1$
- Increment total return  
 $S(s) \leftarrow S(s) + G_t$
- Value is estimated by mean return  $V(s) = S(s)/N(s)$
- By law of large numbers,  
 $V(s) \rightarrow V_\pi(s)$  as  $N(s) \rightarrow \infty$

# First-Visit/Every-Visit Monte-Carlo Policy Evaluation

Input: a policy  $\pi$  to be evaluated

Initialize:

$V(s) \in \mathbb{R}$ , arbitrarily, for all  $s \in \mathcal{S}$

$Returns(s) \leftarrow$  an empty list, for all  $s \in \mathcal{S}$

Loop forever (for each episode):

Generate an episode following  $\pi$ :  $S_0, A_0, R_1, S_1, A_1, R_2, \dots, S_{T-1}, A_{T-1}, R_T$

$G \leftarrow 0$

Loop for each step of episode,  $t = T-1, T-2, \dots, 0$ :

$G \leftarrow \gamma G + R_{t+1}$

Unless  $S_t$  appears in  $S_0, S_1, \dots, S_{t-1}$ :

Append  $G$  to  $Returns(S_t)$

$V(S_t) \leftarrow \text{average}(Returns(S_t))$

Q: First-visit MC? Every-Visit MC?

Unless  $s_t$  appears in  $s_0, s_1, \dots, s_{t-1}$

# Incremental Monte-Carlo Updates

- **Q: Further improvements?** Recap: The mean  $\mu_1, \mu_2, \dots$  of a sequence  $x_1, x_2, \dots$  can be computed incrementally
- Updates  $s_t$  incrementally after episode  $s_1, a_1, r_1, \dots$ ,
- For each state  $s_t$  with return  $G_t$

$$N(s_t) \leftarrow N(s_t) + 1$$

$$V(s_t) \leftarrow V(s_t) + \frac{1}{N(s_t)} (G_t - V(s_t))$$

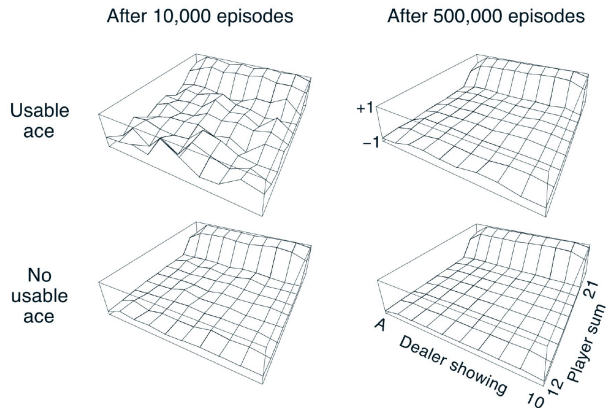
- In **non-stationary problems**, it can be useful to track a running mean, *i.e.*, forget old episodes

$$V(s_t) \leftarrow V(s_t) + \alpha (G_t - V(s_t))$$

## Example: Blackjack (S. & B. Ch5, Page 93)

- Goal: Obtain cards the sum of whose numerical values is as great as possible without exceeding 21. All face cards count as 10, and an ace can count either as 1 or as 11.
- Action stick: Stop receiving cards (and terminate)
- Action draw: Take another card (no replacement)
- Reward for stick: +1/0/−1 if sum of cards  $> / = / <$  sum of dealer cards
- Reward for draw: −1 for sum of cards  $> 21$  (and terminate), 0 otherwise
- Transitions: automatically draw if the sum of cards  $< 12$
- **Q: How many states?** State (200 of them)
  - Current sum (12-21)
  - Dealers showing card (ace-10)
  - Do I have an “usable” ace (yes-no) (count as 11 without going bust)

# Example: Blackjack (Value Function after MC Learning)



**Policy:** stick if the sum of cards  $\geq 20$ , otherwise draw

# Advantage of MC over DP

- MC works when the environment is **unknown**
- Working with a sampled episode has a huge advantage, even when one has complete knowledge of the environment's dynamics, for example, **transition probability is complex to compute**
- Cost of estimating a single state's value **is independent of** the total number of states. We can sample an episode starting from the state of interest, then average returns.

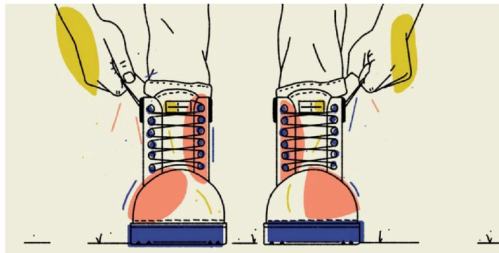
# Monte Carlo Estimation of Action Values

- If a model is not available, then it is particularly useful to estimate **action values** (the values of state-action pairs) rather than **state** values
- The **first-visit** or **every-visit** MC methods converge quadratically to the true expected values as the number of visits to each state-action pair approaches infinity.
- **Complication**: Many state-action pairs may never be visited.
- **Exploring starts**: By specifying that the episodes state in a state-action pair, and that **every pair has a nonzero probability of being selected as the start**.



- 1 Introduction
- 2 Monte-Carlo Learning
- 3 Temporal Difference Learning**
- 4  $TD(\lambda)$

# Bootstrapping



- **Bootstrapping**: To lift oneself up by one's bootstraps
- In RL, bootstrapping means **“using an estimated value in the update step for the same kind of estimated value”**

# Temporal-Difference (TD) Learning

- TD methods learn directly from episodes of experience
- TD is **model-free**: no knowledge of MDP transitions/rewards
- TD learns from incomplete episodes, by **bootstrapping**
- TD updates a guess towards a guess
- Recall: simplest every-visit **Monte Carlo method**:

$$V(s_t) \leftarrow V(s_t) + \alpha[G_t - V(s_t)]$$

Target: the actual return after time  $t$

- The simplest Temporal-Difference method TD(0):

$$V(s_t) \leftarrow V(s_t) + \alpha [r_{t+1} + \gamma V(s_{t+1}) - V(s_t)]$$

Target: an estimate of the return

# MC and TD

- **Goal:** Learn  $V_\pi$  online from experience under policy  $\pi$
- Incremental every-visit Monte-Carlo
- Update value  $V(s_t)$  toward **actual return**  $G_t$

$$V(s_t) \leftarrow V(s_t) + \alpha(G_t - V(s_t))$$

- Simplest temporal-difference learning algorithm: TD(0)
  - Update value  $V(s_t)$  towards **estimated return**  $r_{t+1} + \gamma V(s_{t+1})$

$$V(s_t) \leftarrow V(s_t) + \alpha[r_{t+1} + \gamma V(s_{t+1}) - V(s_t)]$$

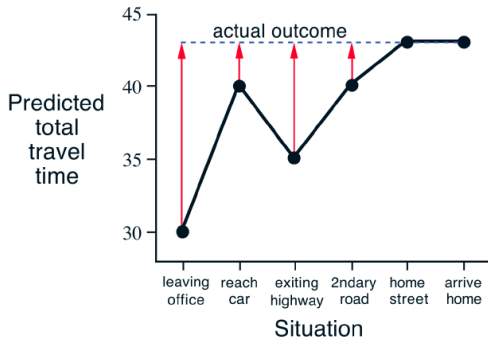
- $r_{t+1} + \gamma V(s_{t+1})$  is called the **TD target**
- $\delta_t = r_{t+1} + \gamma V(s_{t+1}) - V(s_t)$  is called the **TD error**

## Example: Driving Home (S. & B. Ch6, Page 122)

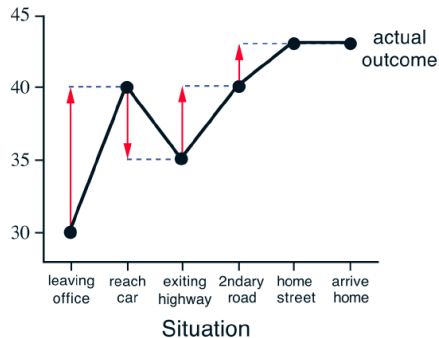
| State                     | Elapsed Time (minutes) | Predicted Time to Go | Predicted Total Time |
|---------------------------|------------------------|----------------------|----------------------|
| leave office              | 0                      | 30                   | 30                   |
| reach car, raining        | 5                      | 35                   | 40                   |
| exit highway              | 20                     | 15                   | 35                   |
| 2ndary road, behind truck | 30                     | 10                   | 40                   |
| entering home street      | 40                     | 3                    | 43                   |
| arrive home               | 43                     | 0                    | 43                   |

# Example: Driving Home (S. & B. Ch6, Page 122)

Changes recommended by  
Monte Carlo methods ( $\alpha = 1$ )



Changes recommended by  
TD methods ( $\alpha = 1$ )



# Advantages and Disadvantages of MC vs. TD

- TD can learn before **knowing** the final outcome
  - TD can learn online after every step
  - MC must wait until the end of the episode before the return is known
- TC can learn **without** the final outcome
  - TD can learn from incomplete sequences
  - MC can only learn from complete sequences
  - TD works in continuing (non-terminating) environments
  - MC only works for episodic (terminating) environments

# Bias/Variance Trade-off

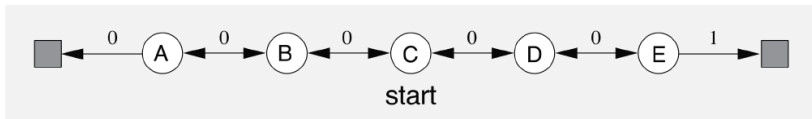
- Return  $G_t = r_t + \gamma r_{t+1} + \dots + \gamma^{T-t} r_{t+T}$  is **unbiased** estimate of  $V_\pi(s_t)$
- True TD target  $r_t + \gamma V_\pi(s_{t+1})$  is **unbiased** estimation of  $V_\pi(s_t)$
- TD target  $r_{t+1} + \gamma V(s_{t+1})$  is **biased** estimate of  $V_\pi(s_t)$
- TD target is much lower variance than the return:
  - Return depends on **many** random actions, transitions, rewards
  - TD target depends on **one** random action, transition, reward



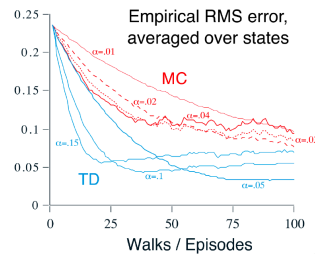
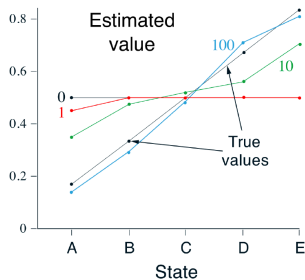
# Advantages and Disadvantages of MC vs. TD

- MC has high variance, zero bias
  - Good convergence properties (even with function approximation)
  - Not very sensitive to initial value
  - Very simple to understand and use
- TD has low variance, some bias
  - Usually more efficient than MC
  - TD(0) converges to  $V_{\pi}(s)$  (but not always with function approximation)
  - More sensitive to initial value

# Example: Random Walk (S. & B. Ch6, Page 125)



MRP: True values of all the states, A through E, are  $\frac{1}{6}, \frac{2}{6}, \frac{3}{6}, \frac{4}{6}, \frac{5}{6}$



# Batch MC and TD

- MC and TD converge:  $V(s) \rightarrow V_{\pi}(s)$  as experience  $\rightarrow \infty$
- But what about a batch solution for finite experience?

$$s_{t+1}^1, a_{t+1}^1, r_{t+1}^1, \dots, r_{t+T_1}^1$$

$$s_{t+1}^2, a_{t+1}^2, r_{t+1}^2, \dots, r_{t+T_2}^2$$

$$\vdots$$

$$s_{t+1}^K, a_{t+1}^K, r_{t+1}^K, \dots, r_{t+T_K}^K$$

- e.g., Repeat sample episode  $k \in [1, K]$

# Example: AB (S. & B. Ch6, Page 127)

- Two states A, B; no discounting; 8 episodes of experience  
     $\{A, 0, B, 0\}$   
     $\{B, 1\}$   
     $\{B, 1\}$   
     $\{B, 1\}$   
     $\{B, 1\}$   
     $\{B, 1\}$   
     $\{B, 1\}$   
     $\{B, 0\}$
- Q: What is  $V(A), V(B)$ ?

# MC: Minimize Mean Square Errors

$\{A, 0, B, 0\}$

$\{B, 1\}$

$\{B, 1\}$

$\{B, 1\}$

$\{B, 1\}$

$\{B, 1\}$

$\{B, 1\}$

$\{B, 0\}$

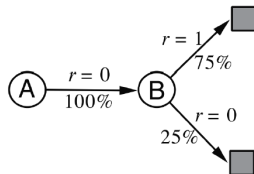
- MC converges to the solution with the **minimum mean-squared error**
  - Best fit to the observed returns

$$\sum_{k=1}^K \sum_{t=1}^{T_k} \left( G_t^k - V(s_t^k) \right)^2$$

- In the AB sample,  $V(A) = 0$

# TD: Certainty Equivalence

$\{A, 0, B, 0\}, \{B, 1\}, \{B, 1\}, \{B, 1\}, \{B, 1\}, \{B, 1\},$   
 $\{B, 1\}, \{B, 0\}$



- TD(0) converges to the solution of max likelihood Markov model
  - Solution to the MDP  $(\mathbf{S}, \mathbf{A}, \mathbf{P}, R, \gamma)$  that best fits the data

$$p(s'|s, a) = \frac{1}{N(s, a)} \sum_{k=1}^K \sum_{t=1}^{T_k} \mathbf{1}(s_t^k, a_t^k, s_{t+1}^k = s, a, s')$$

$$r(s, a) = \frac{1}{N(s, a)} \sum_{k=1}^K \sum_{t=1}^{T_k} \mathbf{1}(s_t^k, a_t^k = s, a)$$

- In the AB sample,  $V(A) = 0.75$

# TD: Certainty Equivalence

TD(0) converges to the solution of **max likelihood Markov model**

- This is correct for the maximum likelihood estimate of a Markov model generating the data
- *i.e.*, if we do **a best fit Markov model, and assume it is exactly correct, and then compute what it predicts.**
- This is called the **certainty-equivalence estimate.**

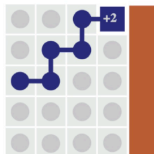
# Advantages and Disadvantages of MC vs. TD

- TD **exploits** the Markov property
  - Usually more efficient in Markov environments
- MC **does not** exploit the Markov property
  - Usually more effective in non-Markov environments



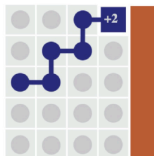
# MC vs. TD (One More Example)

MC



Sometimes the agent reaches its goal.

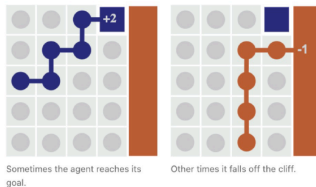
TD



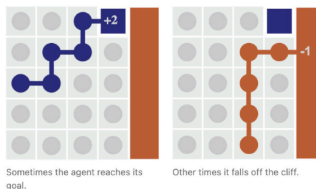
Sometimes the agent reaches its goal.

# MC vs. TD (One More Example)

MC

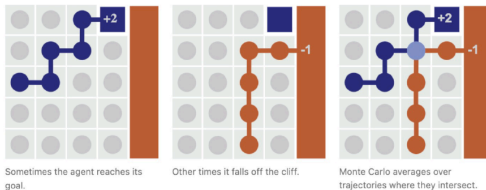


TD

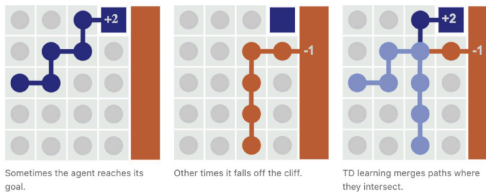


# MC vs. TD (One More Example)

MC

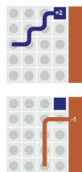


TD

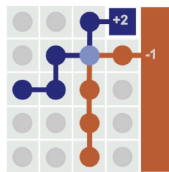


# MC vs. TD (One More Example)

MONTE CARLO ESTIMATION



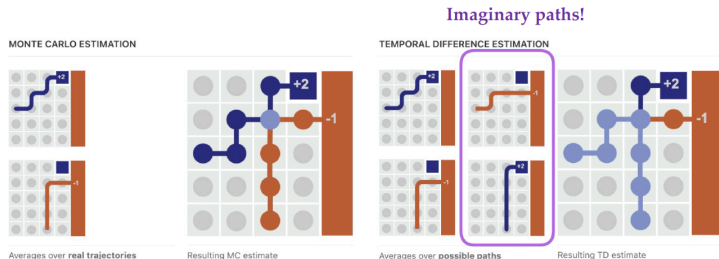
Averages over real trajectories



Resulting MC estimate

$$\text{MC: } V_{S_t} \leftarrow \underbrace{r_t^i}_{\text{Reward from present path}} + \underbrace{\gamma r_{t+1}^i}_{\text{Reward from present path}} + \underbrace{\gamma^2 r_{t+2}^i}_{\text{Reward from present path}} + \dots$$

# MC vs. TD (One More Example)



$$\begin{aligned}
 \text{MC: } V_{S_t} &\leftarrow \underbrace{r_t^i}_{\text{Reward from present path}} + \underbrace{\gamma r_{t+1}^i}_{\text{Reward from present path}} + \underbrace{\gamma^2 r_{t+2}^i}_{\text{Reward from present path}} + \dots \\
 \text{TD: } V_{S_t} &\leftarrow \underbrace{r_t}_{\text{Reward from present path}} + \underbrace{\gamma \mathbb{E}[r'_{t+1}]}_{\text{Expectation over paths intersecting present path}} + \underbrace{\gamma^2 \mathbb{E}[r''_{t+2}]}_{\text{Expectation over paths intersecting path intersecting present path}} + \dots
 \end{aligned}$$

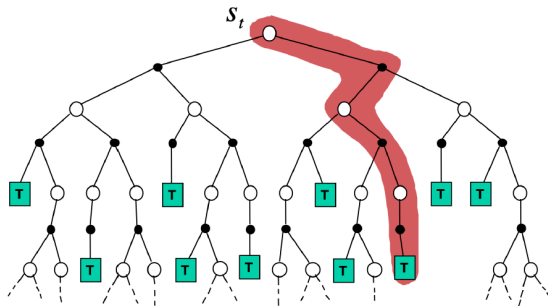
# DP vs. MC vs. TD Learning

$$\begin{aligned} V_{\pi}(s) &= \mathbb{E}_{\pi} [G_t | s_t = s] \\ &= \mathbb{E}_{\pi} \left[ \sum_{k=0}^{\infty} \gamma^k r_{t+k+1} | s_t = s \right] \\ &= \mathbb{E}_{\pi} \left[ r_{t+1} + \gamma \sum_{k=0}^{\infty} \gamma^k r_{t+k+2} | s_t = s \right] \\ &= \mathbb{E}_{\pi} [r_{t+1} + \gamma V_{\pi}(s_{t+1}) | s_t = s] \end{aligned}$$

- **MC**: Sample average return approximates expectation
- **TD**: combine both: Sample expected values and use a current estimate  $V(s_{t+1})$  of the true  $V_{\pi}(s_{t+1})$
- **DP**: the expected values are provided by a model. But we use a current estimate  $V(s_{t+1})$  of the true  $V_{\pi}(s_{t+1})$

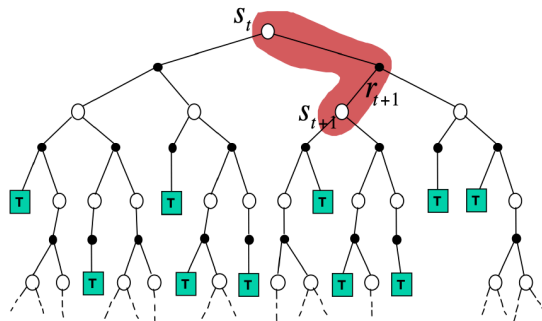
# Monte-Carlo Backup

$$V(s_t) \leftarrow V(s_t) + \alpha (G_t - V(s_t))$$



# Temporal-Difference Backup

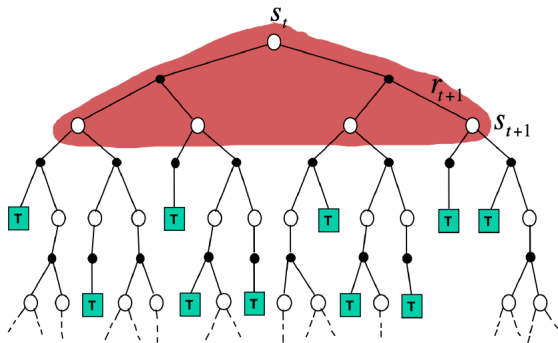
$$V(s_t) \leftarrow V(s_t) + \alpha (r_t + \gamma V(s_{t+1}) - V(s_t))$$





# Dynamic Programming Backup

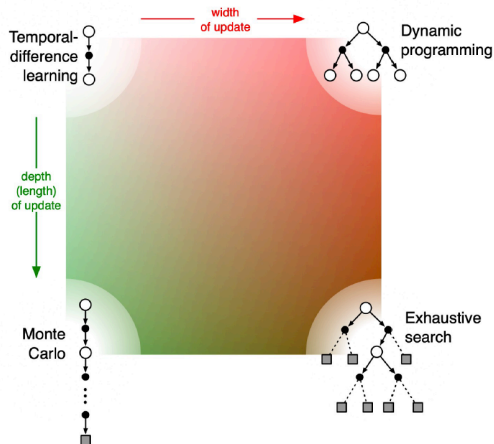
$$V(s_t) \leftarrow \mathbb{E}_{\pi} [r_t + \gamma V(s_{t+1})]$$



# Bootstrapping and Sampling

- **Bootstrapping**: update involves an estimate
  - MC does not bootstrap
  - DP bootstraps
  - TD bootstraps
- **Sampling**: update samples an expectation
  - MC samples
  - DP does not sample
  - TD sample

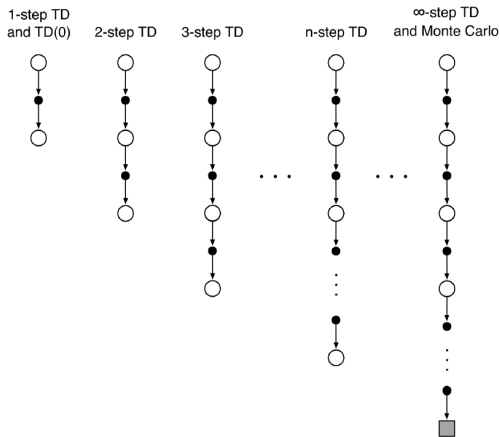
# Unified View of Reinforcement Learning



- 1 Introduction
- 2 Monte-Carlo Learning
- 3 Temporal Difference Learning
- 4 TD( $\lambda$ )

# $n$ -Step Prediction

Let TD target look  $n$  steps into the future



# $n$ -Step Return

- Consider the following  $n$ -step returns for  $n = 1, 2, \dots, \infty$

$$n = 1 \quad (\text{TD})$$

$$G_t^{(1)} = r_t + \gamma V(s_{t+1})$$

$$n = 2$$

$$G_t^{(2)} = r_t + \gamma r_{t+1} + \gamma^2 V(s_{t+2})$$

$$\vdots$$

$$\vdots$$

$$n = \infty \quad (\text{MC})$$

$$G_t^{(\infty)} = r_t + \gamma r_{t+1} + \gamma^{T-1} V(s_{t+T})$$

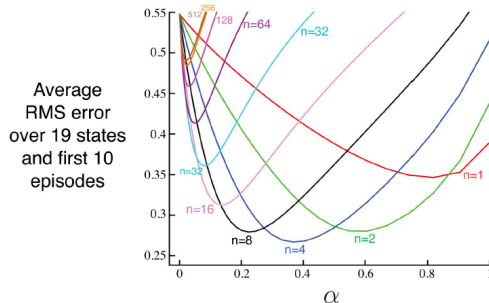
- Define the  $n$ -step return

$$G_t^{(n)} = r_t + \gamma r_{t+1} + \dots + \gamma^{n-1} r_{t+n-1} + \gamma^n V(s_{t+n})$$

- Thus,  $n$ -step temporal-difference learning

$$V(s_t) \leftarrow V(s_t) + \alpha \left( G_t^{(n)} - V(s_t) \right)$$

# Example: Large Random Walk



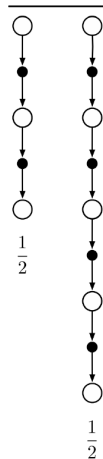
This illustrates how the generalization of TD and Monte Carlo methods to  $n$ -step methods can potentially perform better than either of the two extreme methods.

# Averaging $n$ -Step Returns

- We can average  $n$ -step returns over different  $n$
- e.g., average the 2-step and 4-step returns

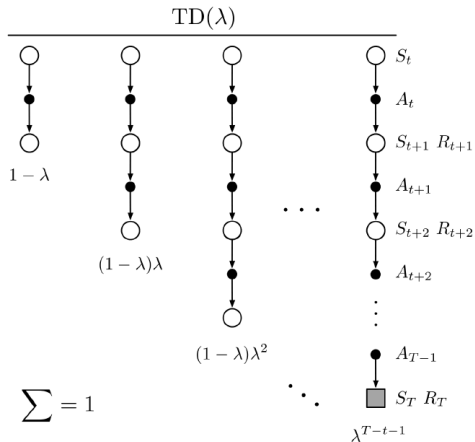
$$\frac{1}{2}G^{(2)} + \frac{1}{2}G^{(4)}$$

- Combines information from two different time-steps
- Q: Can we efficiently combine information from all time-steps?





# $\lambda$ -Return



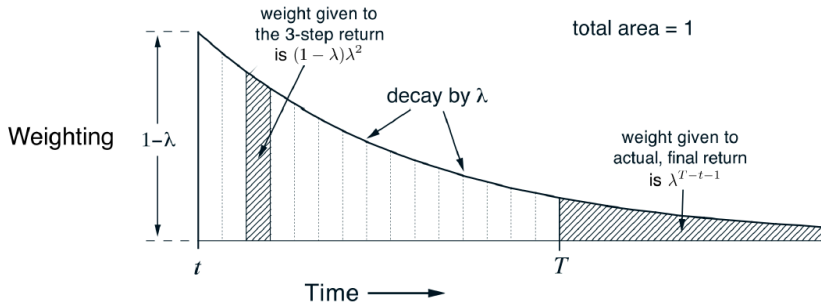
- The  $\lambda$ -return  $G_t^\lambda$  combines all  $n$ -step returns  $G_t^{(n)}$
- Using weight  $(1-\lambda)\lambda^{n-1}$

$$G_t^\lambda = (1-\lambda) \sum_{n=1}^{\infty} \lambda^{n-1} G_t^{(n)}$$

- Forward-view TD( $\lambda$ )

$$V(s_t) \leftarrow V(s_t) + \alpha \left( G_t^\lambda - V(s_t) \right)$$

# TD( $\lambda$ ) Weighted Function



$$G_t^\lambda = (1 - \lambda) \sum_{n=1}^{\infty} \lambda^{n-1} G_t^{(n)}$$

# Extra Reading Materials

- Chapter 5, Monte Carlo Methods, Reinforcement Learning: An Introduction, Sutton & Barto
- Chapter 6, Temporal-Difference Learning, Reinforcement Learning: An Introduction, Sutton & Barto